

Contents

1	テーブル定義書 (メイン)	2
1.1	1. 文書概要	2
1.1.1	1.1 目的	2
1.1.2	1.2 対象範囲	2
1.1.3	1.3 版数	2
1.1.4	1.4 関連ドキュメント	2
1.2	2. テーブル一覧	2
1.2.1	2.1 retention_class 値の正本定義 (audit_logs.retention_class)	4
1.2.2	2.2 主要関係性 (§ 7.3 から移管)	5
1.2.3	2.3 メンバー権限保持方式 (設計判断履歴、 § 7.1.1 から移管)	5
1.3	3. テーブル詳細	5
1.3.1	3.1 accounts(アカウント)	5
1.3.2	3.2 account_project_grants(オーナー / メンバープロジェクト割当 + プロジェクト別 ロール)	7
1.3.3	3.4 sessions(セッション)	8
1.3.4	3.5 access_tokens(アクセストークン)	8
1.3.5	3.6 projects(プロジェクト)	9
1.3.6	3.7 project_legacy_keys(レガシー API キー)	9
1.3.7	3.8 allowed_domains(許可ドメイン)	9
1.3.8	3.8a project_ip_allowlist(プロジェクト IP 許可リスト)	10
1.3.9	3.8b project_notification_settings(廃止)	10
1.3.10	3.8c project_notification_recipients(廃止)	10
1.3.11	3.9 faqs(FAQ)	10
1.3.12	3.10 faq_revisions(FAQ 改訂履歴)	11
1.3.13	3.11 faq_search_fts(FAQ 全文検索、FTS5 仮想テーブル)	11
1.3.14	3.12 question_logs(質問ログ)	11
1.3.15	3.13 inquiries(案件)	12
1.3.16	3.14 inquiry_status_history(案件状態履歴)	13
1.3.17	3.15 inquiry_contacts(エンドユーザー連絡先、PII 暗号化)	13
1.3.18	3.16 chat_rooms(個別チャット部屋)	13
1.3.19	3.17 chat_messages(チャットメッセージ)	14
1.3.20	3.18 notification_logs(通知ログ)	14
1.3.21	3.19 ai_threshold_persistent_cache(AI しきい値キャッシュ)	15
1.3.22	3.20 email_suppression_list(メールサプレスリスト、全契約横断)	15
1.3.23	3.21 audit_logs(監査ログ、メイン側)	15
1.3.24	3.22 error_logs(エラーログ)	16
1.3.25	3.23 billing_subscriptions(課金サブスクリプション)	16
1.3.26	3.24 billing_invoices(請求書、7 年保持)	16
1.3.27	3.25 usage_metering(利用量計測)	17
1.3.28	3.26 owner_quota_overrides(契約別レート / 予算上書き、統一実装)	17
1.3.29	3.27 service_announcements(お知らせ Control Plane)	17
1.3.30	3.28 announcement_recipients(お知らせ配信先 Control Plane)	18
1.3.31	3.29 inbox_messages(受信箱 Tenant Plane)	18
1.3.32	3.30 withdrawal_requests(退会申請)	19
1.3.33	3.31 terms_versions(規約版数、全契約横断)	19
1.3.34	3.32 terms_agreements(規約同意)	19
1.4	4. ER 図 (主要部分、 § 7.2 から移管)	19
1.5	5. テーブル間関連 (§ 2.2 と同期)	20
1.6	6. 命名規則・型方針 (§ 7.5 から移管)	20
1.7	7. SaaS データ分離観点	20
1.7.1	7.1 オーナー境界によるデータ分離方式 (§ 2.6 + § 7.1 から移管)	20
1.7.2	7.2 オーナー行とメンバー行の判定	20
1.7.3	7.3 全契約横断テーブル (owner_account_id を持たない)	20
1.8	8. 状態コード値・状態遷移 (§ 4.x から移管)	20
1.8.1	8.1 FAQ 状態遷移 (faqs.status)	20
1.8.2	8.2 案件状態遷移 (inquiries.case_status、共有概念正本)	21
1.8.3	8.3 チャット部屋自動クローズ 6 段階 (chat_rooms.reminder_state)	21
1.8.4	8.4 通知状態遷移 (notification_logs.delivery_state)	21
1.8.5	8.5 契約状態遷移 (accounts.contract_status、共有概念正本)	22

1.9	9. データ削除モード (§ 7.8 から移管、v1.10 で論理削除モード追加)	22
1.9.1	9.1 削除モード対応表 (v1.10 改訂)	22
1.9.2	9.2 論理削除の物理削除バッチ (v1.10、DD14 正本)	23
1.10	10. データ保持の調整制約 (NFR-702)	23
1.11	11. PII 取扱 (§ 7.10 から移管)	23
1.12	12. インデックス方針 (§ 7.6 から移管)	23
1.13	13. 未決事項	24
1.14	14. 変更履歴	24

1 テーブル定義書 (メイン)

1.1 1. 文書概要

1.1.1 1.1 目的

メインシステムのデータベース (Cloudflare D1) に存在するすべてのテーブルについて、カラム / 型 / 制約 / インデックス / 外部キー / コード値 / 保持期間 / 保持起点 / **owner_account_id** 分離方式を一元化する。

1.1.2 1.2 対象範囲

- 対象: 利用者向けデータ 32 テーブル (認証 / プロジェクト / FAQ / 案件 / チャット / 通知 / 課金 / お知らせ / 監査 / 規約 等)
- 対象外: 運営者側専用テーブル (**operator_***, **audit_logs**(ハッシュチェーン構造の運営者部分), **web-hook_events**, **operator_approvals**, **accounts_retired** 等)—— 02_ 運営者システム/個別設計書群/04_ テーブル定義書.md 参照

1.1.3 1.3 版数

項目	値
版数	1.0
更新日	2026-05-17

1.1.4 1.4 関連ドキュメント

ドキュメント名	役割	参照先
索引	11 ドキュメント体系の俯瞰	00_ 索引.md
共有概念対応表	accounts.contract_status / オーナーユーザーモデルなど	../共有/共有概念.md
API 設計書	テーブルを操作する API	03_API 設計書.md
認証・認可設計書	オーナー境界判定	09_ 認証認可設計書.md
権限設計書	account_project_grants.role(3 ロール定義)	05_ 権限設計書.md
セキュリティ設計書	PII 暗号化 / 監査 / 削除モード	10_ セキュリティ設計書.md
課金・請求設計書	accounts.contract_status 正本 / 契約上書きテーブル	11_ 課金請求設計書.md
詳細設計書	実装関連の詳細 (モジュール構成 / バッチ / マイグレーション)	../03_ 詳細設計書.md

1.2 2. テーブル一覧

#	テーブル名	論理名	概要	主キー	owner_account_id	主管 FR	保持期間	保持起点	retention_class
1	accounts	アカウント	オーナー / メンバー両方を 1 表に保持 (is_owner=1 がオーナー)	id	自己参照 (オーナー行は id = owner_account_id)	FR-015a FR-33d	永久 (契約存続中)/ 退会後は § 7 削除ポリシー	作成日時	billing

#	テーブル名	論理名	概要	主キー	owner_account	主管FR	保持期間	保持起点	retention_class
2	account_project_grants	プロジェクト割当+プロジェクト別ロール	メンバーのプロジェクトアクセスとロール(admin / member)	(accounts project 経由)	(accounts 経由)	FR-015, FR-030-035	紐づくアカウントに従属	-	general
4	sessions	セッション	セッション管理(複数デバイス対応)	id	(accounts 経由)	FR-022, FR-332	期限切れ後 90 日	最終アクティビティ	general
5	access_tokens	アクセストークン	招待 / パスワード再設定 / 再入室等のトークン	id	(accounts 経由)	FR-004, FR-008	使用後 30 日	発行日時	general
6	projects	プロジェクト	FAQ プロジェクト / ウィジェット設定	id	あり	FR-030-035	永久(プロジェクト存続中)	作成日時	general
7	project_widget_keys	ウィジェット API キー	ウィジェット ローテーション 30 日猶予	id	(projects 経由)	-	30 日	ローテーション日時	general
8	allowed_domains	許可ドメイン	ウィジェット埋め込み許可ドメイン	id	(projects 経由)	FR-150-156	プロジェクトに従属	-	general
8a	project_ip_restrictions	プロジェクト IP 許可リスト	FAQ ウィジェット向け IP 制限(プロジェクト単位 / FR-179・FR-330)	id	(projects 経由)	FR-179, FR-330	プロジェクトに従属	-	general
8b	project_notification_settings	プロジェクト通知設定	SCR-030 を廃止しプロジェクト関連通知を常時 ON 固定としたため、本テーブルは不要	—	—	—	—	—	
8c	project_notification_recipients	プロジェクト通知受信者	SCR-030 をプロジェクト単位トグルに簡素化したため不要。配信先は当該プロジェクトの account_project_grants を配信時に動的スキャンして決定	—	—	—	—	—	
9	faqs	FAQ	FAQ 本体	id	あり	FR-040-048, FR-322	永久 (FAQ 存続中)	作成日時	general
10	faq_revisions	FAQ 改訂履歴	全文スナップショット 50 件保持	id	(faqs 経由)	FR-322	永久(版ごと)	改訂日時	general
11	faq_search_index	FAQ 全文検索	FTS5 仮想テーブル (trigram)	(rowid)	(faqs 連動)	FR-300	faqs に従属	-	-
12	question_logs	質問ログ	エンドユーザーからの質問 / AI 推論	id	あり	FR-003, NFR-601	730 日 (自動匿名化)	投稿日時	general
13	question_faq_refs	参照 FAQ	質問と FAQ の M:N 関係	id	(経由)	-	question_logs に従属	-	general
14	inquiries	案件	未解決質問のグループ	id	あり	FR-070-079	永久 (case_status=closed後は NFR-706)	作成日時 / クローズ日時	general

#	テーブル名	論理名	概要	主キー	owner_account	主管FR	保持期間	保持起点	retention_class
15	inquiry_status	案件状態履歴	状態変更トレース	id	(inquiries 経由)	FR-077	inquiries に従属	-	general
16	inquiry_contact	案件連絡先	エンドユーザーメール (AES-256-GCM 暗号化 + HMAC)	id	(inquiries 経由)	FR-079, NFR-319	inquiries に従属 (物理削除 / 匿名化)	-	general
17	chat_rooms	個別チャット部屋	エンドユーザーとの個別チャット (自動クローズ 6 段階)	id	(inquiries 経由)	FR-080-091	730 日 (自動匿名化)	クローズ日時と最終投稿日時の遅い方	general
18	chat_messages	個別チャットメッセージ	個別チャットの投稿	id	(chat_rooms 経由)	FR-080-091	chat_rooms に従属	-	general
19	notification_log	通知ログ	メール通知の送信履歴	id	あり	NFR-506, NFR-319	1 年	送信日時	general
20	ai_thresholds	AI 検知閾値キャッシュ	3 階層しきい値の永続キャッシュ	id	あり (scope=owner 時)	FR-341	30 日	更新日時	general
21	email_suppression_list	メーリングリスト	バウンス・苦情の全契約横断管理	id	(全契約横断)	FR-149b	永久	サプレス日時	general
22	audit_logs	監査ログ	メイン側 API 操作ログ + 監査 (運営者側がハッシュチェーン構造の正本)	id	あり (owner_account カラム)	NFR-602a, 602b	retention_class	操作日時	general / billing / operator_high_priv
23	error_logs	エラーログ	サーバーエラー記録	id	あり (可)	NFR-704	180 日	発生日時	general
24	billing_subscription	課金サブスクリプション	Stripe Subscription 連動	id	あり	FR-148	永久 (契約継続中)	作成日時	billing
25	billing_invoices	請求書	月次請求書 (電子帳簿保存法)	id	あり	FR-148, NFR-602b	7 年	発行日時	billing
26	usage_metrics	利用量計測	質問数 / FAQ 件数 / チャット部屋数	id	あり	FR-148	7 年	計測日時	billing
27	owner_quotes	契約別 Overrides / 予算上書き	運営者上書き (SCR-093) — レート / 予算ともに本テーブルで統一実装 (resource_kind で区別)	id	あり	IF #5	5 年	設定日時	operator_high_priv
28	service_announcements	お知らせ (Control Plane)	運営者発信のお知らせ本体	id	(全契約横断 or 限定)	-	5 年	配信日時	operator_high_priv
29	announcement_recipients	お知らせ受信者	お知らせ配信先・配信集計・監査	id	あり	-	5 年	配信日時	general
30	inbox_messages	受信箱 (Tenant Plane)	利用者のお知らせ・通知の既読管理	id	あり	FR-182-192	1 年	既読日時	general
31	withdrawal_requests	退会申請	退会申請レコード (30 日猶予)	id	あり	FR-189	7 年	申請日時	billing
32	terms_versions	規約版数	規約 / プライバシーポリシーの版	version	(全契約横断)	-	永久	制定日時	general
33	terms_agreements	規約同意	利用者ごとの同意履歴	id	(accounts 経由)	-	7 年	同意日時	billing

1.2.1 2.1 retention_class 値の正本定義 (audit_logs.retention_class)

値	保持期間	用途	例
general	1年(NFR-602a)	業務監査	通常の操作 / チャット / FAQ 編集 / ログイン
billing	7年(NFR-602b、電子帳簿保存法)	課金・法定保管	請求 / 支払 / サブスク / 予算 / owner:suspend prefix
operator_high (5年(似))	5年(NFR-602d、SOX 類似)	運営者高権限操作	復元 / 契約停止 / 物理削除 / AI 変更 / レート上書き / ローテーション

詳細は [10_セキュリティ設計書.md §7](#)(運営者側 §7 が正本) を参照。

1.2.2 2.2 主要関係性 (§7.3 から移管)

関係	型	説明
オーナー ☒ アカウント	1:N	契約削除時にアカウントも削除対象(オーナー行: is_owner=1)
契約 ☒ プロジェクト	1:N	-
プロジェクト ☒ FAQ	1:N	FAQ はプロジェクトに帰属
プロジェクト ☒ 未解決質問	1:N	-
未解決質問 ☒ チャット部屋	1:1	-
チャット部屋 ☒ メッセージ	1:N	-
未解決質問 ☒ FAQ	0..1:N	faqs.source_unresolved_question_id で参照可
お知らせ ☒ 受信箱	1:N	service_announcements → announcement_recipients → inbox_messages の fan-out

1.2.3 2.3 メンバー権限保持方式 (設計判断履歴、§7.1.1 から移管)

採用方式は **プロジェクト単位ロール方式**(`account_project_grants.role`)。(`account_id`, `project_id`) 複合 PK + `role` 列 (`admin` / `member`) + `granted_at` / `granted_by` 保持で行レベル監査を維持。

v1.10 でオーナー (`accounts.is_owner=1`) も `account_project_grants` に行を持つ(プロジェクト作成時に `account_project_grants(account_id= オーナー, project_id= 新 PJ, role='admin', valid=1)` を自動INSERT)。従来の「オーナーは行を持たない暗黙の admin」設計は廃止した。認可フロー先頭の `isOwner=true` bypass は維持し、`account_project_grants` 行は通知宛先解決 (critical 通知) と画面表示 (プロジェクト管理者一覧) の事実情報として参照する。MVP 前期に検討した「個別権限フラグ方式 (`account_permissions` 別テーブル + `permission_kind` 5 値)」は、運用パターンが固定的なため UI / 認可ロジック双方の複雑さに見合わず廃止した。

v1.10 で `valid` カラムによる論理削除を全削除対象テーブルに導入 (対象テーブルは §9 削除モード表を正本)。`valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1))` を追加し、論理削除は `UPDATE ... SET valid=0` で表現する。論理削除データは `updated_at` 基準で 90 日経過後に物理削除バッチで物理削除する ([DD14_バッチ・非同期処理.md](#) を正本)。`deleted_at` カラムは追加せず、論理削除日時は既存 `updated_at`(`valid` が 0 に更新された時刻) で代用する。例外テーブル: `audit_logs`(tombstone 方式と整合)、`sessions`(`revoked_at` で代用)、法定保管 7 年テーブル (`billing_invoices` / `usage_metering` / `withdrawal_requests` / `terms_versions` / `terms_agreements`)、全契約横断テーブル (`email_suppression_list`)、ログ系 (`notification_logs` / `error_logs` / `inbox_messages`)、トークン / 履歴系 (`access_tokens` / `faq_revisions` / `faq_search_fts` / `chat_messages` / `inquiry_status_history` / `question_log_faq_refs`)。

1.3 3. テーブル詳細

1.3.1 3.1 accounts(アカウント)

項目	内容
テーブル名	accounts
論理名	アカウント
概要	オーナー (<code>is_owner=1</code>) とメンバー (<code>is_owner=0</code>) の両方を 1 表に保持
主キー	id(TEXT, ULID)

項目	内容
設計判断	オーナー行は owner_account_id = id で自己参照。v1.10 でオーナー (is_owner=1) も account_project_grants に role='admin' 行を保持する (プロジェクト作成時に自動 INSERT。認可上は isOwner=true bypass を優先する)。論理削除は valid カラムで表現 (valid=0 = 論理削除済、valid=1 = 有効)。論理削除日時は updated_at で代用 (deleted_at 列は MVP 前期に契約レベルの退会日時用に追加されたものを継続利用するが、v1.10 で新規追加した valid カラムとは独立)

1.3.1.1 DDL(詳細設計書 行 5827-5859 由来)

```

CREATE TABLE accounts (
  id TEXT PRIMARY KEY,
  owner_account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  email_encrypted TEXT NOT NULL,
  email_hmac TEXT NOT NULL,
  password_hash TEXT,
  name TEXT,
  role TEXT NOT NULL CHECK (role IN ('admin','service_operator','end_user')),
  is_owner INTEGER NOT NULL DEFAULT 0 CHECK (is_owner IN (0,1)),
  status TEXT NOT NULL DEFAULT 'pending_verification'
  CHECK (status IN ('pending_verification','pending_activation','active')),
  contract_status TEXT NOT NULL DEFAULT 'active'
  CHECK (contract_status IN ('active','suspended','deleted_pending','deleted')),
  data_deletion_mode TEXT NOT NULL DEFAULT 'physical'
  CHECK (data_deletion_mode IN ('physical','anonymize')),
  valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  last_login_at TEXT,
  email_verified_at TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  deleted_at TEXT,
  CHECK ((is_owner = 1 AND owner_account_id = id) OR (is_owner = 0 AND owner_account_id <> id));
-- v1.10: 同一オーナー配下で valid=1 のメール重複のみ禁止 (論理削除済みの valid=0 行は重複許容、同メールで)
CREATE UNIQUE INDEX uq_accounts_owner_email ON accounts(owner_account_id, email_hmac) WHERE valid = 1;
CREATE INDEX idx_accounts_email_hmac ON accounts(email_hmac);
CREATE INDEX idx_accounts_owner_role_status ON accounts(owner_account_id, role, status);
CREATE INDEX idx_accounts_is_owner ON accounts(is_owner) WHERE is_owner = 1;
CREATE INDEX idx_accounts_valid ON accounts(valid) WHERE valid = 0;

```

1.3.1.2 コード値: role / is_owner / status / contract_status

カラム	値	説明
role	admin	利用者管理者
role	service_operator	運営者(運営者画面アクセス用、メインでも accounts に格納)
role	end_user	エンドユーザー(ウィジェット利用者)
is_owner	0	メンバー
is_owner	1	オーナー(契約あたり1つ)
status	pending_verification	メール未確認
status	pending_activation	招待中(初回パスワード未設定)
status	active	有効
contract_status	active	有効(オーナー行でのみ意味を持つ。共有概念正本)
contract_status	suspended	サスペンション中
contract_status	deleted_pending	退会申請中(30日猶予)
contract_status	deleted	削除済み(v1.10でオーナー退会完了時は valid=0 も同時セット)
valid	1	有効(v1.10)
valid	0	論理削除済み(v1.10。論理削除日時は updated_at で判別。updated_at < now() - 90d AND valid=0 の行は物理削除バッチで物理削除される)

詳細状態遷移は [10_課金・請求設計.md §4](#) を正本とする。

1.3.1.3 contract_status × valid の整合 (v1.10 新設)

is_owner	contract_status	valid	状態
1	active / suspended	1	オーナー稼働中
1	deleted_pending	1	オーナー退会申請中 (30 日猶予)
1	deleted	0	オーナー退会完了 (valid=0 と必ず同時セット)
0	active(デフォルト)	1	メンバー有効
0	active(デフォルト)	0	メンバー論理削除済み (contract_status は触らない、valid=0 のみ)

1.3.2 3.2 account_project_grants(オーナー / メンバープロジェクト割当 + プロジェクト別ロール)

オーナー(`accounts.is_owner=1`)とメンバー(`is_owner=0`)の両方に対し、プロジェクトごとに `admin`(プロジェクト管理者)/`member`(メンバー)のロールを付与する。**v1.10** でオーナーも本テーブルに行を持つ(プロジェクト作成時に (`account_id=` オーナー, `project_id=` 新 PJ, `role='admin'`, `valid=1`) 行を自動 INSERT)。認可フロー先頭の `isOwner=true` bypass は維持し、本テーブルは通知宛先解決と画面表示に使う。

割当ゼロ状態の発生条件 (v1.7 で FR-015d 不変条件を撤廃 / v1.10 で論理削除化): メンバー削除を **アカウント論理削除** (`UPDATE accounts SET valid=0`) に統一し (v1.7 物理削除統一を v1.10 で論理削除へ再統一)、`account_project_grants` 行のみの単独削除(離脱)操作は提供しない。論理削除に伴い `account_project_grants` 行も `valid=0` に同時 UPDATE する。`valid=1` 行が 0 行残った状態は構造的に発生しない(アカウント論理削除と同時に発生するため)。プロジェクト削除起因で `valid=1` 割当が 0 件になるメンバーは同一トランザクション内で `accounts.valid=0` に論理削除する(下記「プロジェクト削除時の孤立メンバー処理」)。最後の `admin` の有無に関しては、v1.10 でオーナーが常に `admin` 行を保持するため「プロジェクトに `admin` 行が 0 件になる」状態は構造的に発生しない(**E-AUTHZ-LAST-ADMIN-PROTECTED** の判定はオーナー行除外、メンバー由来の `admin` のみで判定)。

1.3.2.1 DDL

```
CREATE TABLE account_project_grants (
  account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  project_id TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  role       TEXT NOT NULL CHECK (role IN ('admin','member')),
  valid      INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  granted_at TEXT NOT NULL,
  granted_by TEXT NOT NULL REFERENCES accounts(id),
  updated_at TEXT NOT NULL,
  PRIMARY KEY (account_id, project_id)
);
-- v1.10: 同一 (account_id, project_id) ペアで valid=1 行は 1 件のみ、再招待時は valid=0 履歴行を残し
-- 物理 PK は (account_id, project_id) のままだが、論理削除済の同ペアを保持する目的では新規行を作成せず
CREATE INDEX idx_account_project_grants_project ON account_project_grants(project_id);
-- プロジェクト管理者の高速検索用 (critical 通知宛先解決でも使用、valid=1 の admin のみ)
CREATE INDEX idx_account_project_grants_role_admin
  ON account_project_grants(account_id) WHERE role = 'admin' AND valid = 1;
CREATE INDEX idx_account_project_grants_valid ON account_project_grants(valid) WHERE valid = 0;
```

PK と再招待: (`account_id`, `project_id`) の物理 PK は維持。同じメンバー × 同じプロジェクトの再招待は、既存行の `valid` を 0 → 1 に切り替える UPSERT(`INSERT ... ON CONFLICT(account_id, project_id) DO UPDATE SET valid=1, role=?, granted_at=now(), granted_by=?, updated_at=now()`) で表現する。これにより、同一ペアで `valid=1` 行は最大 1 件となり、部分 UNIQUE インデックスは不要。論理削除済み (`valid=0`) 行の `granted_at` は最初の招待時刻を維持し、再招待時に上書きしない仕様としても良いが、MVP では復活時に最新値で上書きする(監査ログ側で履歴を担保)。

1.3.2.2 コード値: role 2 値

値	表示名 (UI)	該当プロジェクト範囲で可能な操作
admin	プロジェクト管理者	FAQ 管理 / 個別チャット対応 / ログ参照 / ユーザー管理 (該当プロジェクトへの招待・離脱・ロール変更)
member	メンバー	FAQ 管理 / 個別チャット対応 / ログ参照

詳細は [05_権限設計書.md §3](#) を正本とする。

1.3.2.3 プロジェクト削除時の孤立メンバー処理 (アプリ層、v1.10 で論理削除化) v1.10 でプロジェクト削除と孤立メンバー cleanup を論理削除に統一。次の処理はアプリ層のトランザクション内で明示する:

1. `UPDATE account_project_grants SET valid=0, updated_at=now() WHERE project_id=? AND valid=1`(オーナー自身の admin 行も含む全行)
2. UPDATE 後に「他プロジェクト割当 (`valid=1` 行) が 0 件かつ `is_owner=0`」となったメンバー `accounts` 行を抽出して `UPDATE accounts SET valid=0, updated_at=now() WHERE id IN (...)` および全セッション失効 (`UPDATE sessions SET revoked_at=now()`)
3. `UPDATE projects SET status='deleted', valid=0, deleted_at=now(), updated_at=now() WHERE id=?`
4. 関連テーブル(`allowed_domains / project_ip_allowlist / faqs / inquiries / inquiry_contacts / chat_rooms / question_logs`)の対象行を `valid=0` に伝播

オーナーは `accounts` 本体の論理削除対象外 (`is_owner=1` は常に `valid=1` 維持)。当該プロジェクトに対する `account_project_grants` 行は `valid=0` に論理削除する。論理削除データは `updated_at < now() - 90d AND valid=0` の物理削除バッチ (DD14) で 90 日後に物理削除される。物理削除時に `accounts` の物理 DELETE が走ると `ON DELETE CASCADE` で関連行も連鎖物理削除されることに留意。詳細は [DD02_プロジェクト・FAQ 管理.md](#) を参照。

1.3.3 3.4 sessions(セッション)

1.3.3.1 DDL(行 5894-5905)

```
CREATE TABLE sessions (  
  id TEXT PRIMARY KEY,  
  account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,  
  ip_address TEXT NOT NULL,  
  user_agent TEXT,  
  created_at TEXT NOT NULL,  
  last_accessed_at TEXT NOT NULL,  
  expires_at TEXT NOT NULL,  
  revoked_at TEXT  
);  
CREATE INDEX idx_sessions_account ON sessions(account_id) WHERE revoked_at IS NULL;  
CREATE INDEX idx_sessions_expires ON sessions(expires_at) WHERE revoked_at IS NULL;
```

1.3.4 3.5 access_tokens(アクセストークン)

1.3.4.1 DDL(行 5908-5920)

```
CREATE TABLE access_tokens (  
  id TEXT PRIMARY KEY,  
  account_id TEXT REFERENCES accounts(id) ON DELETE CASCADE,  
  token_hash TEXT NOT NULL,  
  purpose TEXT NOT NULL CHECK (purpose IN ('email_verify', 'password_reset', 'activation', 're  
  meta TEXT,  
  created_at TEXT NOT NULL,  
  expires_at TEXT NOT NULL,  
  used_at TEXT  
);  
CREATE UNIQUE INDEX uq_access_tokens_hash ON access_tokens(token_hash);  
CREATE INDEX idx_access_tokens_expires ON access_tokens(expires_at) WHERE used_at IS NULL;
```

1.3.4.2 コード値: purpose

値	用途
email_verify	メール確認

値	用途
password_reset	パスワード再設定
activation	招待受諾(初回パスワード設定)
reentry	エンドユーザー再入室

1.3.5 3.6 projects(プロジェクト)

1.3.5.1 DDL(行 5923-5944)

```
CREATE TABLE projects (
  id TEXT PRIMARY KEY,
  owner_account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  name TEXT NOT NULL CHECK (length(name) BETWEEN 1 AND 100),
  description TEXT,
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'de
widget_public_key TEXT NOT NULL UNIQUE,
widget_public_key_expires_at TEXT NOT NULL,
contact_email TEXT,
contact_email_verified_at TEXT,
settings TEXT NOT NULL DEFAULT '{}',
valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
created_at TEXT NOT NULL,
updated_at TEXT NOT NULL,
deleted_at TEXT,
CHECK (contact_email_verified_at IS NULL OR contact_email IS NOT NULL),
-- v1.10: status='deleted' のとき必ず valid=0(両者は同時セット)
CHECK ((status = 'active' AND valid = 1) OR (status = 'deleted' AND valid = 0))
);
CREATE INDEX idx_projects_owner_status ON projects(owner_account_id, status);
CREATE INDEX idx_projects_widget_key ON projects(widget_public_key);
CREATE INDEX idx_projects_valid ON projects(valid) WHERE valid = 0;
```

1.3.6 3.7 project_legacy_keys(レガシー API キー)

1.3.6.1 DDL(行 5947-5954)

```
CREATE TABLE project_legacy_keys (
  id TEXT PRIMARY KEY,
  project_id TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  public_key TEXT NOT NULL,
  grace_until TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_legacy_keys_key ON project_legacy_keys(public_key);
```

1.3.7 3.8 allowed_domains(許可ドメイン)

1.3.7.1 DDL(行 5957-5963)

```
CREATE TABLE allowed_domains (
  id TEXT PRIMARY KEY,
  project_id TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  domain TEXT NOT NULL,
  valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL DEFAULT (datetime('now'))
);
-- v1.10: valid=1 行のみ UNIQUE(再追加可能にするため部分 UNIQUE 化)
CREATE UNIQUE INDEX uq_allowed_domains_project_domain ON allowed_domains(project_id, domain) WH
CREATE INDEX idx_allowed_domains_valid ON allowed_domains(valid) WHERE valid = 0;
```

1.3.8 3.8a project_ip_allowlist(プロジェクト IP 許可リスト)

FAQ ウィジェット (エンドユーザーアクセス) に対する IP 制限を CIDR で保持する。プロジェクト単位設定。空 = 制限なし。管理画面 API は本テーブルの参照対象外 (FR-179 / FR-330)。

1.3.8.1 DDL

```
CREATE TABLE project_ip_allowlist (  
  id TEXT PRIMARY KEY,  
  project_id TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,  
  cidr TEXT NOT NULL,  
  family TEXT NOT NULL CHECK (family IN ('ipv4','ipv6')), -- IPv4 / IPv6  
  note TEXT, -- 評価高速化用に  
  valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)), -- 任意メモ (SCR  
  created_at TEXT NOT NULL DEFAULT (datetime('now')),  
  updated_at TEXT NOT NULL DEFAULT (datetime('now'))  
);  
-- v1.10: valid=1 行のみ UNIQUE(再追加可能にするため部分 UNIQUE 化)  
CREATE UNIQUE INDEX uq_project_ip_allowlist_project_cidr ON project_ip_allowlist(project_id, cidr)  
CREATE INDEX idx_project_ip_allowlist_project_family ON project_ip_allowlist(project_id, family)  
CREATE INDEX idx_project_ip_allowlist_valid ON project_ip_allowlist(valid) WHERE valid=1
```

1.3.8.2 制約・評価

- ・ 1 プロジェクトあたり最大 100 行 (アプリ層で検証)
- ・ 同一プロジェクト内の重複 CIDR は UNIQUE 制約で拒否
- ・ ウィジェット API ハンドラは (project_id, family) インデックスを用いてキャッシュ生成し、判定はメモリ上の LPM(longest-prefix-match) で 5 分 TTL キャッシュ
- ・ 監査ログ: audit_logs.action に project.ip_allowlist.update を記録 (変更前後の CIDR セット差分を metadata に格納)

1.3.9 3.8b project_notification_settings(廃止)

v2.0 で SCR-030 プロジェクト設定画面を廃止し、プロジェクト関連通知 (NOTIF-CHAT_REPLY_TO_STAFF / NOTIF-CHAT_HOLD_CHECK) を常時 ON 固定としたため、本テーブルは不要となり廃止。配信先は配信時点で当該プロジェクトの account_project_grants を動的スキャンしてオーナー + admin + member 全員に解決する (画面設定の参照は不要)。critical は引き続き共有概念に基づき強制送信、契約単位通知は契約 WS 側で扱う。

1.3.10 3.8c project_notification_recipients(廃止)

v1.8 で SCR-030 をプロジェクト単位の通知オン/オフトグルに簡素化したため本テーブルは不要となり廃止。配信先は account_project_grants を配信時に動的スキャンして決定する (オーナー + admin + member 全員)。critical は引き続き共有概念に基づきオーナー + 同一オーナースコープで account_project_grants.role='admin' を 1 件でも保持する全プロジェクト管理者へ強制送信する (本テーブル/トグルの影響を受けない)。

1.3.11 3.9 faqs(FAQ)

1.3.11.1 DDL(行 5966-5987)

```
CREATE TABLE faqs (  
  id TEXT PRIMARY KEY,  
  owner_account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,  
  project_id TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,  
  title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 200),  
  body TEXT NOT NULL CHECK (length(body) BETWEEN 1 AND 10000),  
  category TEXT,  
  status TEXT NOT NULL DEFAULT 'draft'  
  CHECK (status IN ('draft','published','hidden','deleted')),  
  version INTEGER NOT NULL DEFAULT 1,  
  tags TEXT,  
  source_unresolved_question_id TEXT REFERENCES inquiries(id),
```

```

valid                INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
created_by           TEXT NOT NULL REFERENCES accounts(id),
updated_by           TEXT NOT NULL REFERENCES accounts(id),
created_at           TEXT NOT NULL,
updated_at           TEXT NOT NULL,
deleted_at           TEXT
);
CREATE INDEX idx_faqs_owner_project ON faqs(owner_account_id, project_id);
CREATE INDEX idx_faqs_project_status ON faqs(project_id, status);
CREATE INDEX idx_faqs_source_inquiry ON faqs(source_unresolved_question_id);
CREATE INDEX idx_faqs_category ON faqs(project_id, category) WHERE category IS NOT NULL;
CREATE INDEX idx_faqs_valid ON faqs(valid) WHERE valid = 0;

```

1.3.11.2 コード値: **status**(状態遷移は §8 参照)

値	説明
draft	下書き
published	公開中
hidden	非公開
deleted	論理削除 (valid=0 と同時セット。復元は 運営者のみ = IF #4)

valid は v1.10 で全テーブル共通の論理削除フラグとして追加された (**status='deleted'** のとき必ず **valid=0**、それ以外は **valid=1**)。プロジェクト削除に伴うカスケード論理削除では **status** を変えずに **valid=0** のみセットすることがあるが、UI/API では **valid=0** の行を常に除外するため利用者には見えない。

1.3.12 3.10 **faq_revisions**(FAQ 改訂履歴)

1.3.12.1 DDL(行 5992-6008)

```

CREATE TABLE faq_revisions (
  id                TEXT PRIMARY KEY,
  faq_id            TEXT NOT NULL REFERENCES faqs(id) ON DELETE CASCADE,
  version           INTEGER NOT NULL,
  title             TEXT NOT NULL,
  body              TEXT NOT NULL,
  category          TEXT,
  tags              TEXT,
  status_snapshot   TEXT NOT NULL CHECK (status_snapshot IN ('draft','published','hidden','deleted')),
  source            TEXT NOT NULL DEFAULT 'manual' CHECK (source IN ('manual','import')),
  created_at        TEXT NOT NULL,
  created_by        TEXT NOT NULL REFERENCES accounts(id)
);
CREATE INDEX idx_faq_revisions_faq ON faq_revisions(faq_id, version DESC);
CREATE INDEX idx_faq_revisions_source ON faq_revisions(source);

```

1.3.13 3.11 **faq_search_fts**(FAQ 全文検索、FTS5 仮想テーブル)

1.3.13.1 DDL(行 6011-6017)

```

CREATE VIRTUAL TABLE faq_search_fts USING fts5(
  title, body, category, tags,
  content='faqs',
  content_rowid='rowid',
  tokenize='trigram'
);

```

faqs テーブルへの INSERT / UPDATE / DELETE トリガで自動同期。詳細は詳細設計 § 9.9 参照。

1.3.14 3.12 **question_logs**(質問ログ)

1.3.14.1 DDL(行 6020-6042)

```

CREATE TABLE question_logs (
  id TEXT PRIMARY KEY,
  owner_account_id TEXT NOT NULL REFERENCES accounts(id),
  project_id TEXT NOT NULL REFERENCES projects(id),
  user_question TEXT NOT NULL CHECK (length(user_question) BETWEEN 1 AND 2000),
  ai_response TEXT,
  is_resolved INTEGER NOT NULL DEFAULT 0,
  metering_billable INTEGER NOT NULL DEFAULT 0,
  confidence_score REAL,
  relevance_score REAL,
  ai_model TEXT,
  ai_token_count_input INTEGER,
  ai_token_count_output INTEGER,
  result_type TEXT CHECK (result_type IN ('answered','unanswered','error')),
  result_reason_code TEXT,
  pii_masked INTEGER NOT NULL DEFAULT 0,
  session_id TEXT,
  ip_address TEXT,
  valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_qlog_owner_created ON question_logs(owner_account_id, created_at DES
CREATE INDEX idx_qlog_project_billable_created ON question_logs(project_id, metering_billable,
CREATE INDEX idx_qlog_valid ON question_logs(valid) WHERE valid = 0;

```

1.3.15 3.13 inquiries(案件)

1.3.15.1 DDL(行 6053-6074)—— 詳細設計の case_status CHECK 制約欠落を補正

```

CREATE TABLE inquiries (
  id TEXT PRIMARY KEY,
  owner_account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  project_id TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  inquiry_code TEXT NOT NULL,
  question_log_id TEXT REFERENCES question_logs(id),
  user_question TEXT NOT NULL,
  case_status TEXT NOT NULL DEFAULT 'open'
  CHECK (case_status IN ('open','resolved','closed','faq_registered'))
  faq_candidate_status TEXT NOT NULL DEFAULT 'none'
  CHECK (faq_candidate_status IN ('none','candidate','drafted','regist
  faq_draft TEXT,
  assignee_account_id TEXT REFERENCES accounts(id) ON DELETE SET NULL,
  valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  closed_at TEXT,
  deleted_at TEXT
);
CREATE INDEX idx_inquiries_contract_status_created ON inquiries(owner_account_id, case_status,
CREATE INDEX idx_inquiries_project_status ON inquiries(project_id, case_status);
CREATE INDEX idx_inquiries_assignee ON inquiries(assignee_account_id) WHERE case
CREATE UNIQUE INDEX uq_inquiries_code ON inquiries(inquiry_code);
CREATE INDEX idx_inquiries_valid ON inquiries(valid) WHERE valid = 0;

```

1.3.15.2 コード値: case_status(共有概念正本 = FR-079 自動 closed 遷移禁止)

値	説明	遷移条件
open	未対応	新規作成時
resolved	解決済み	admin の「解決済み」操作のみ
closed	対応不要終了	admin の明示操作のみ。自動 retention で closed に遷移させない (FR-079)

値	説明	遷移条件
faq_registered	FAQ 化済み	admin の「FAQ 登録完了」操作

詳細遷移は § 8.2 参照。case_status と valid は独立した概念で、closed は admin による明示クローズ(業務終了)、valid=0 はプロジェクト削除に伴うカスケード論理削除(v1.10)。両者は別軸として共存する。

1.3.16 3.14 inquiry_status_history(案件状態履歴)

1.3.16.1 DDL(行 6076-6086)

```
CREATE TABLE inquiry_status_history (
  id          TEXT PRIMARY KEY,
  inquiry_id  TEXT NOT NULL REFERENCES inquiries(id) ON DELETE CASCADE,
  from_status TEXT NOT NULL,
  to_status   TEXT NOT NULL,
  reason      TEXT,
  changed_by  TEXT REFERENCES accounts(id),
  changed_at  TEXT NOT NULL
);
CREATE INDEX idx_isth_inquiry_changed ON inquiry_status_history(inquiry_id, changed_at DESC);
```

1.3.17 3.15 inquiry_contacts(エンドユーザー連絡先、PII 暗号化)

1.3.17.1 DDL(行 6089-6100)

```
CREATE TABLE inquiry_contacts (
  id          TEXT PRIMARY KEY,
  inquiry_id  TEXT NOT NULL REFERENCES inquiries(id) ON DELETE CASCADE,
  email_encrypted TEXT,
  email_hmac   TEXT,
  display_name TEXT,
  verified     INTEGER NOT NULL DEFAULT 0,
  notify_enabled INTEGER NOT NULL DEFAULT 1,
  valid        INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  created_at   TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at   TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX idx_inquiry_contacts_email_hmac ON inquiry_contacts(email_hmac);
CREATE UNIQUE INDEX uq_inquiry_contacts_inquiry ON inquiry_contacts(inquiry_id);
CREATE INDEX idx_inquiry_contacts_valid ON inquiry_contacts(valid) WHERE valid = 0;
```

email_encrypted は AES-256-GCM(オーナー派生) で暗号化。email_hmac は HMAC-SHA256 で検算用。詳細は [10_セキュリティ設計書.md § 3](#) 参照。

1.3.18 3.16 chat_rooms(個別チャット部屋)

1.3.18.1 DDL(行 6103-6121)

```
CREATE TABLE chat_rooms (
  id          TEXT PRIMARY KEY,
  inquiry_id  TEXT NOT NULL REFERENCES inquiries(id) ON DELETE CASCADE,
  room_status TEXT NOT NULL DEFAULT 'open' CHECK (room_status IN ('open','closed')),
  reminder_state TEXT NOT NULL DEFAULT 'active'
                                CHECK (reminder_state IN ('active','stage1_pending_admin','stage2_user_no_response','stage3_user_no_response','stage4_final_no_response','stage5_final_no_response','stage6_auto_response')),
  last_message_at TEXT,
  last_message_actor_type TEXT,
  reminder_state_changed_at TEXT,
  valid        INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),
  created_at   TEXT NOT NULL,
  updated_at   TEXT NOT NULL DEFAULT (datetime('now')),
  closed_at    TEXT
```

```
);
CREATE INDEX idx_chat_rooms_inquiry ON chat_rooms(inquiry_id);
CREATE INDEX idx_chat_rooms_open_reminder ON chat_rooms(reminder_state, last_message_at) WHERE
CREATE INDEX idx_chat_rooms_valid ON chat_rooms(valid) WHERE valid = 0;
```

1.3.18.2 コード値: **reminder_state**(自動クローズ 6 段階) 詳細は § 8.3 参照。

1.3.19 3.17 chat_messages(チャットメッセージ)

1.3.19.1 DDL(行 6124-6132)

```
CREATE TABLE chat_messages (
  id TEXT PRIMARY KEY,
  chat_room_id TEXT NOT NULL REFERENCES chat_rooms(id) ON DELETE CASCADE,
  actor_type TEXT NOT NULL CHECK (actor_type IN ('admin', 'end_user', 'system')),
  actor_id TEXT REFERENCES accounts(id) ON DELETE SET NULL,
  content TEXT NOT NULL CHECK (length(content) BETWEEN 1 AND 2000),
  created_at TEXT NOT NULL
);
CREATE INDEX idx_chat_messages_room_created ON chat_messages(chat_room_id, created_at);
```

1.3.20 3.18 notification_logs(通知ログ)

1.3.20.1 DDL(行 6241-6260)

```
CREATE TABLE notification_logs (
  id TEXT PRIMARY KEY,
  owner_account_id TEXT NOT NULL,
  inquiry_id TEXT,
  chat_message_id TEXT,
  recipient_email_hmac TEXT,
  notification_type TEXT NOT NULL,
  delivery_state TEXT NOT NULL
  CHECK (delivery_state IN ('queued', 'sending', 'sent', 'delivered',
                           'failed', 'bounced', 'complained', 'suppressed'
                           'message_id TEXT,
  attempt_count INTEGER NOT NULL DEFAULT 0,
  sent_at TEXT,
  delivered_at TEXT,
  failed_at TEXT,
  fail_reason TEXT,
  created_at TEXT NOT NULL
);
CREATE INDEX idx_nlog_owner_created ON notification_logs(owner_account_id, created_at DESC);
CREATE INDEX idx_nlog_message_id ON notification_logs(message_id);
CREATE INDEX idx_nlog_state ON notification_logs(delivery_state);
```

1.3.20.2 コード値: **delivery_state**(状態遷移は §8.4 参照)

値	説明
queued	Queue 投入済み
sending	送信中
sent	送信成功
delivered	配信成功 (Resend Webhook 経由)
failed	失敗 (当方再送対象、最大 3 回)
bounced	バウンス (ハードまたはソフト 5 連続)
complained	スパム報告
suppressed	サプレスリスト追加済み (終了状態)

1.3.21 3.19 ai_threshold_persistent_cache(AI しきい値キャッシュ)

1.3.21.1 DDL(行 6323-6335)

```
CREATE TABLE ai_threshold_persistent_cache (  
  id TEXT PRIMARY KEY,  
  owner_account_id TEXT,  
  project_id TEXT,  
  scope TEXT NOT NULL CHECK (scope IN ('global','owner','project')),  
  confidence_threshold REAL NOT NULL,  
  relevance_threshold REAL NOT NULL,  
  version INTEGER NOT NULL,  
  received_at TEXT NOT NULL,  
  updated_at TEXT NOT NULL  
);
```

```
CREATE UNIQUE INDEX uq_ai_threshold_scope ON ai_threshold_persistent_cache(scope, COALESCE(owne
```

3 階層しきい値: global / owner / project。詳細は [03_API 設計書.md §5.5.5](#)(IF #6) 参照。

1.3.22 3.20 email_suppression_list(メールサプレスリスト、全契約横断)

1.3.22.1 DDL(行 6289-6298)

```
CREATE TABLE email_suppression_list (  
  id TEXT PRIMARY KEY,  
  email_hmac TEXT NOT NULL UNIQUE,  
  reason TEXT NOT NULL CHECK (reason IN ('bounced_hard','bounced_soft_5x','complained',''),  
  is_permanent INTEGER NOT NULL DEFAULT 1,  
  created_at TEXT NOT NULL,  
  released_at TEXT  
);
```

```
CREATE INDEX idx_suppression_permanent ON email_suppression_list(is_permanent, email_hmac);
```

全契約横断テーブル (R-003 共通ドメインレピュテーション保護対策)。 [10_セキュリティ設計書.md §11](#) 参照。

1.3.23 3.21 audit_logs(監査ログ、メイン側)

1.3.23.1 DDL(行 6266-6287)

```
CREATE TABLE audit_logs (  
  id TEXT PRIMARY KEY,  
  owner_account_id TEXT,  
  actor_account_id TEXT,  
  actor_role TEXT,  
  action TEXT NOT NULL,  
  target_type TEXT,  
  target_id TEXT,  
  ip_address_masked TEXT,  
  user_agent TEXT,  
  metadata TEXT,  
  retention_class TEXT NOT NULL DEFAULT 'general'  
  CHECK (retention_class IN ('general','billing','operator_high_priv')),  
  prev_hash TEXT,  
  current_hash TEXT NOT NULL,  
  tombstone INTEGER NOT NULL DEFAULT 0,  
  created_at TEXT NOT NULL  
);  
CREATE INDEX idx_audit_owner_action_created ON audit_logs(owner_account_id, action, created_at)  
CREATE INDEX idx_audit_retention_created ON audit_logs(retention_class, created_at);
```

- ・ハッシュチェーン構造の正本(運営者側 §7)。prev_hash / current_hash で連結
- ・3 区分保持 (general 1y / billing 7y / operator_high_priv 5y)
- ・保持期間経過後は tombstone 方式 (本文を物理削除し、log_id / prev_hash / current_hash / deleted_at を保持してチェーン破断を防ぐ)

1.3.24 3.22 error_logs(エラーログ)

1.3.24.1 DDL(行 9052-6061、詳細設計内記載)

```
CREATE TABLE error_logs (  
  id TEXT PRIMARY KEY,  
  owner_account_id TEXT,  
  account_id TEXT,  
  url TEXT,  
  error_type TEXT,  
  stack TEXT,  
  occurred_at TEXT NOT NULL  
);  
CREATE INDEX idx_error_logs_owner_occurred ON error_logs(owner_account_id, occurred_at DESC);
```

PII / トークン / カード情報を記録禁止 (FR-114、エンドユーザー入力のは先頭 100 文字 + ハッシュのみ)。

1.3.25 3.23 billing_subscriptions(課金サブスクリプション)

1.3.25.1 DDL(行 6138-6154)

```
CREATE TABLE billing_subscriptions (  
  id TEXT PRIMARY KEY,  
  owner_account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,  
  stripe_subscription_id TEXT UNIQUE,  
  status TEXT NOT NULL  
  CHECK (status IN ('trialing', 'active', 'past_due', 'canceled', 'unpaid',  
  plan_id TEXT NOT NULL,  
  trial_started_at TEXT,  
  trial_ends_at TEXT,  
  current_period_start TEXT,  
  current_period_end TEXT,  
  cancel_at TEXT,  
  valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)),  
  created_at TEXT NOT NULL,  
  updated_at TEXT NOT NULL  
);  
CREATE INDEX idx_billing_subs_owner ON billing_subscriptions(owner_account_id);  
CREATE INDEX idx_billing_subs_status ON billing_subscriptions(status);  
CREATE INDEX idx_billing_subs_valid ON billing_subscriptions(valid) WHERE valid = 0;
```

v1.10 物理削除バッチ除外: billing_subscriptions は valid カラムを持つが、電子帳簿保存法 / 法定保管要件との整合のため、90 日後の物理削除バッチ (DD14) の対象外とする。valid=0 行は永久に保持し、Stripe 連動の取消履歴として参照可能にする。

1.3.26 3.24 billing_invoices(請求書、7 年保持)

1.3.26.1 DDL(行 6156-6174)

```
CREATE TABLE billing_invoices (  
  id TEXT PRIMARY KEY,  
  owner_account_id TEXT NOT NULL REFERENCES accounts(id),  
  billing_year_month TEXT NOT NULL,  
  status TEXT NOT NULL CHECK (status IN ('draft', 'issued', 'paid', 'past_due', 'refun  
  amount_total INTEGER NOT NULL,  
  amount_tax INTEGER NOT NULL DEFAULT 0,  
  currency TEXT NOT NULL DEFAULT 'JPY',  
  stripe_invoice_id TEXT UNIQUE,  
  pdf_r2_key TEXT,  
  issued_at TEXT,  
  paid_at TEXT,  
  refunded_at TEXT,  
  created_at TEXT NOT NULL  
);
```

```
CREATE UNIQUE INDEX uq_billing_invoices_owner_month ON billing_invoices(owner_account_id, billi
CREATE INDEX idx_billing_invoices_status ON billing_invoices(status);
```

1.3.27 3.25 usage_metering(利用量計測)

1.3.27.1 DDL(行 6176-6190)

```
CREATE TABLE usage_metering (
  id TEXT PRIMARY KEY,
  owner_account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  billing_year_month TEXT NOT NULL,
  question_count INTEGER NOT NULL DEFAULT 0,
  faq_count_snapshot INTEGER NOT NULL DEFAULT 0,
  chat_room_count INTEGER NOT NULL DEFAULT 0,
  ai_token_input INTEGER NOT NULL DEFAULT 0,
  ai_token_output INTEGER NOT NULL DEFAULT 0,
  ai_cost_yen INTEGER NOT NULL DEFAULT 0,
  finalized_at TEXT,
  updated_at TEXT NOT NULL
);
CREATE UNIQUE INDEX uq_usage_metering_owner_month ON usage_metering(owner_account_id, billing_y
```

1.3.28 3.26 owner_quota_overrides(契約別レート / 予算上書き、統一実装)

重要: rate_limit_overrides / budget_limit_overrides は別テーブルではなく、本テーブル + re-source_kind 値で統一実装される。

1.3.28.1 DDL(行 6337-6350)

```
CREATE TABLE owner_quota_overrides (
  id TEXT PRIMARY KEY,
  owner_account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  resource_kind TEXT NOT NULL
    CHECK (resource_kind IN ('widget_ask_per_min', 'widget_inquiry_per_min', 'widget_
      'email_per_hour', 'admin_api_per_min')),
  threshold INTEGER NOT NULL,
  window_sec INTEGER NOT NULL,
  valid_until TEXT NOT NULL,
  reason TEXT,
  created_by TEXT,
  created_at TEXT NOT NULL
);
CREATE INDEX idx_tqo_owner_kind ON owner_quota_overrides(owner_account_id, resource_kind, valid
```

1.3.28.2 コード値: resource_kind

値	用途
widget_ask_per_min	ウィジェット質問レート
widget_inquiry_per_min	ウィジェット未解決質問送信レート
widget_chat_per_min	ウィジェットチャット投稿レート
email_per_hour	メール送信レート
admin_api_per_min	管理API レート

詳細は [11_課金請求設計書.md § 3 / 05_権限設計書.md § 7](#) 参照。

1.3.29 3.27 service_announcements(お知らせ Control Plane)

1.3.29.1 DDL(行 6196-6208)

```

CREATE TABLE service_announcements (
  id TEXT PRIMARY KEY,
  title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 200),
  body_html TEXT NOT NULL,
  importance TEXT NOT NULL CHECK (importance IN ('low','normal','high','critical')),
  audience_owner_account_ids TEXT,
  published_at TEXT NOT NULL,
  retracted_at TEXT,
  created_by TEXT NOT NULL,
  created_at TEXT NOT NULL
);
CREATE INDEX idx_announcements_published ON service_announcements(published_at DESC) WHERE retracted_at IS NULL AND importance < 4
importance 4 値は共有概念正本(critical はメール強制送信)。07_メッセージ一覧.md §2.4 参照。

```

1.3.30 3.28 announcement_recipients(お知らせ配信先 Control Plane)

1.3.30.1 DDL(行 6210-6220)

```

CREATE TABLE announcement_recipients (
  id TEXT PRIMARY KEY,
  announcement_id TEXT NOT NULL REFERENCES service_announcements(id) ON DELETE CASCADE,
  owner_account_id TEXT NOT NULL,
  account_id TEXT NOT NULL,
  sent_at TEXT NOT NULL,
  delivery_status TEXT NOT NULL CHECK (delivery_status IN ('pending','delivered','failed')),
  read_at TEXT
);
CREATE INDEX idx_ar_announcement ON announcement_recipients(announcement_id);
CREATE INDEX idx_ar_owner_account ON announcement_recipients(owner_account_id, account_id);

```

1.3.31 3.29 inbox_messages(受信箱 Tenant Plane)

1.3.31.1 DDL(行 6222-6239)

```

CREATE TABLE inbox_messages (
  id TEXT PRIMARY KEY,
  owner_account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  recipient_account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  category TEXT NOT NULL CHECK (category IN ('billing','announcement','system')),
  priority TEXT NOT NULL CHECK (priority IN ('low','normal','high','critical')),
  title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 200),
  body_html TEXT NOT NULL,
  source_kind TEXT,
  source_id TEXT,
  dedup_key TEXT NOT NULL,
  read_at TEXT,
  hidden_at TEXT,
  created_at TEXT NOT NULL
);
CREATE INDEX idx_inbox_owner_account_unread ON inbox_messages(owner_account_id, recipient_account_id) WHERE read_at IS NULL;
CREATE INDEX idx_inbox_dedup ON inbox_messages(owner_account_id, dedup_key, created_at);

```

1.3.31.2 お知らせ二層構成

- **Control Plane**(service_announcements / announcement_recipients): 運営者作成お知らせ本体 + 配信集計・監査
- **Tenant Plane**(inbox_messages): 利用者管理者の受信箱(既読管理)
- **announcement** カテゴリは Control → Tenant の二層経由。billing / system は inbox_messages 直接生成
- 重複生成対策: (owner_account_id, event_kind, dedup_key) で 60 分以内は 1 件に集約

1.3.32 3.30 withdrawal_requests(退会申請)

1.3.32.1 DDL(行 6352-6361)

```
CREATE TABLE withdrawal_requests (  
  id TEXT PRIMARY KEY,  
  owner_account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,  
  applied_at TEXT NOT NULL,  
  applied_by TEXT NOT NULL REFERENCES accounts(id),  
  reason TEXT,  
  scheduled_deletion_at TEXT NOT NULL  
);  
CREATE INDEX idx_withdrawal_owner_scheduled ON withdrawal_requests(owner_account_id, scheduled_at);
```

1.3.33 3.31 terms_versions(規約版数、全契約横断)

1.3.33.1 DDL(行 6300-6308)

```
CREATE TABLE terms_versions (  
  version TEXT PRIMARY KEY,  
  effective_date TEXT NOT NULL,  
  body_html TEXT NOT NULL,  
  diff_summary TEXT,  
  notification_sent_at TEXT,  
  consent_deadline_days INTEGER NOT NULL DEFAULT 14,  
  created_at TEXT NOT NULL  
);
```

1.3.34 3.32 terms_agreements(規約同意)

1.3.34.1 DDL(行 6310-6317)

```
CREATE TABLE terms_agreements (  
  id TEXT PRIMARY KEY,  
  account_id TEXT NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,  
  terms_version TEXT NOT NULL REFERENCES terms_versions(version),  
  agreed_at TEXT NOT NULL,  
  agreed_ip_masked TEXT  
);  
CREATE UNIQUE INDEX uq_terms_account_version ON terms_agreements(account_id, terms_version);
```

1.4 4. ER 図 (主要部分、§7.2 から移管)

```
erDiagram  
  accounts ||--o{ accounts : "owner_account_id (self-ref)"  
  accounts ||--o{ projects : owns  
  accounts ||--o{ billing_subscriptions : has  
  accounts ||--o{ inbox_messages : "1-N(オーナー配下)"  
  accounts ||--o{ audit_logs : "1-N"  
  accounts ||--o{ account_project_grants : "0..N(メンバーのみ、role: admin/member)"  
  accounts ||--o{ sessions : has  
  accounts ||--o{ access_tokens : has  
  accounts ||--o{ terms_agreements : has  
  projects ||--o{ faqs : contains  
  projects ||--o{ question_logs : records  
  projects ||--o{ inquiries : holds  
  projects ||--o{ allowed_domains : authorizes  
  projects ||--o{ project_legacy_keys : rotates  
  faqs ||--o{ faq_revisions : versioned  
  faqs }o--o{ inquiries : "source_unresolved_question_id(opt)"  
  inquiries ||--|| chat_rooms : has  
  inquiries ||--o{ inquiry_contacts : has  
  inquiries ||--o{ inquiry_status_history : tracks
```

```

chat_rooms ||--o{ chat_messages : contains
service_announcements ||--o{ announcement_recipients : "fan-out"
announcement_recipients ||--o{ inbox_messages : "copied to"
billing_subscriptions ||--o{ billing_invoices : generates
accounts ||--o{ usage_metering : metered
accounts ||--o{ owner_quota_overrides : overridden
terms_versions ||--o{ terms_agreements : agreed

```

1.5 5. テーブル間関連 (§2.2 と同期)

§ 2.2 「主要関係性」を参照。

1.6 6. 命名規則・型方針 (§7.5 から移管)

対象	規則
テーブル	snake_case 複数形
主キー	文字列 ID(ULID)/ *_versions 等の例外あり
外部キー	{table_singular}_id
日時	TEXT(ISO 8601 UTC、NFR-1103)
状態	status または {対象}_status
真偽値	INTEGER 0/1
JSON	TEXT 列に JSON 文字列で保存
暗号化	{column}_encrypted(AES-256-GCM) + {column}_hmac(検索用) の 2 列構成

1.7 7. SaaS データ分離観点

1.7.1 7.1 オーナー境界によるデータ分離方式 (§2.6 + §7.1 から移管)

- 全契約スコープのテーブルに `owner_account_id` カラム必須 (全契約横断テーブルを除く — § 7.3 参照)
- API/ 認可レイヤで境界検証: `actor.owner_account_id == target.owner_account_id` (認証・認可設計書 09 § 5)
- 違反時は **404 偽装** (403 だと相手契約のリソース存在が漏れる)
- 150 契約到達時に D1 シャーディング設計を着手予定 (Future 参照)

1.7.2 7.2 オーナー行とメンバー行の判定

- オーナー行: `accounts.is_owner=1` かつ `owner_account_id = id` (自己参照)
- メンバー行: `accounts.is_owner=0` かつ `owner_account_id <> id`
- `accounts` テーブルの CHECK 制約で強制: `((is_owner = 1 AND owner_account_id = id) OR (is_owner = 0 AND owner_account_id <> id))`

1.7.3 7.3 全契約横断テーブル (`owner_account_id` を持たない)

テーブル名	理由
email_suppression_list	メール bounce/ 苦情の全契約横断管理 (R-003 共通ドメイン対策)
terms_versions	規約/ プライバシーポリシーの版 (全利用者共通)
service_announcements	運営者発信のお知らせ本体 (配信先は announcement_recipients で契約別)

1.8 8. 状態コード値・状態遷移 (§4.x から移管)

1.8.1 8.1 FAQ 状態遷移 (faqs.status)

From	To	トリガー	条件
-	draft	FAQ 作成	admin のみ
draft	published	公開操作	admin のみ、interactive セッション、内容確認ガード (基本設計書 § 4.2.1)

From	To	トリガー	条件
draft	hidden	非公開保存	admin のみ
published	hidden	非公開化	admin のみ
hidden	published	再公開	admin のみ
any	deleted	論理削除	admin のみ
deleted	draft	復元	運営者のみ (IF #4)

ガード: draft → published 直接遷移を 3 層で禁止 (API / Domain / DB)。公開状態への遷移は管理者の明示操作のみ。

1.8.2 8.2 案件状態遷移 (inquiries.case_status、共有概念正本)

From	To	トリガー	条件	通知
-	open	未解決質問登録	システム自動	エンドユーザーへ IN-QUIRY_CREATED
open	resolved	admin 「解決済み」	admin のみ	なし
open	closed	admin 「対応不要終了」	admin の明示操作のみ。自動 retention で closed に遷移させない (FR-079)	エンドユーザーへ in-box(announcement)
any	faq_registered	admin 「FAQ 登録完了」	admin のみ	エンドユーザーへ FAQ_REGISTERED

1.8.3 8.3 チャット部屋自動クローズ 6 段階 (chat_rooms.reminder_state)

From	To	トリガー	経過時間	通知
active	stage1_pending_close	最終投稿から 1 日経過	7 日	admin へ CHAT_HOLD_CHECK
stage1_pending_close	stage2_user_check_close	admin 「利用者待ち」判定 or 14 日未操作	即時 / 14 日	エンドユーザーへ CHAT_RESOLUTION_CHECK
stage2_user_check_close	stage3_user_no_response	投稿なし	7 日	なし (Cron 同期)
stage3_user_no_response	stage4_final_check_close	即時遷移	(同周期)	エンドユーザーへ CHAT_FINAL_CONFIRM
stage4_final_check_close	stage5_final_no_response	投稿なし	7 日	なし
stage5_final_no_response	stage6_auto_close	即時遷移	(3 日後)	エンドユーザーへ CHAT_AUTO_CLOSED、room_status=closed
stage6_auto_close	active (再オープン)	エンドユーザー再アクセス	30 日以内	なし
stage6_auto_close	access_tokens (消滅)	30 日経過	-	エンドユーザーへ「新規部屋案内」
任意の active 系	active	EU / admin 投稿	-	カウンタリセット
active 系	手動 close	admin 明示操作	-	エンドユーザーへ CHAT_CLOSED
closed	active	admin 再オープン	30 日以内	監査ログ記録必須

1.8.4 8.4 通知状態遷移 (notification_logs.delivery_state)

From	To	トリガー
-	queued	Queue 投入
queued	sending	Worker 起動
sending	sent	Resend API 成功
sending	failed	Resend API 失敗 (恒久) —— 当方再送対象 (最大 3 回、NFR-506)
sent	delivered	email.delivered Webhook (終了状態)
sent	bounced	email.bounced Webhook (ハード / ソフト 5 連続)
sent	complained	email.complained Webhook

From	To	トリガー
sent	delayed	email.delivery_delayed Webhook(Resend 再送中)
delayed	delivered / bounced	後続結果
bounced	suppressed	サプレスリスト追加
complained	suppressed	即時永久サプレス (復帰不可)
failed	queued	当方再送 (最大 3 回、3 回失敗 + 24 時間経過で admin へ system 通知)

1.8.5 8.5 契約状態遷移 (accounts.contract_status、共有概念正本)

詳細は 11_課金請求設計書.md § 4 / § 5 を正本とする。本書では概要のみ:

From	To	トリガー	reason
-	active	契約作成	-
active	suspended	IF #1(支払い失敗 7 日猶予経過)	payment_failure_grace_expired
active	suspended	IF #1(運営者手動)	operator_manual / tos_violation
active	suspended	Cron(トライアル終了 + 支払方法未登録)	trial_expired_no_payment_method
suspended	active	IF #1(支払再開 / 解除)	-
active / suspended	deleted_pending	admin 退会申請 (SCR-024)	-
deleted_pending	active / suspended(元の状態)	admin 撤回 (30 日以内)	-
deleted_pending	deleted	30 日経過 + 物理削除 Batch	-

1.9 9. データ削除モード (§7.8 から移管、v1.10 で論理削除モード追加)

設定: accounts.data_deletion_mode(physical/anonymize、初期値 physical) で保持 (v1.6 で notification_preferences JSON 内から独立カラムに昇格)。

v1.10 で 第 3 のモード「論理削除」を導入。論理削除は UPDATE … SET valid=0 で行い、90 日経過後の物理削除バッチ (DD14) で物理削除モード (physical/anonymize) に従って実体を消去する。SCR-017-M1 のメンバー削除と SCR-026 のプロジェクト削除は常に論理削除を経由する (v1.10)。

1.9.1 9.1 削除モード対応表 (v1.10 改訂)

対象テーブル	論理削除 (v1.10 新設)	物理削除 (90 日後バッチ or 即時、data_deletion_mode='physical')	匿名化 (data_deletion_mode='anonymize')
accounts	valid=0	DELETE(ON DELETE CASCADE に関連行連鎖)	email_encrypted → anon_<sha256>, name → 「匿名」
account_project_links	valid=0	DELETE(accounts CASCADE 経由)	(匿名化対象外、accounts 連鎖物理削除)
projects	status='deleted', valid=0	DELETE(関連 faqs / inquiries 等は CASCADE 経由)	name → 「削除済みプロジェクト」
allowed_domains	valid=0	DELETE	(匿名化対象外)
project_ip_addresses	valid=0	DELETE	(匿名化対象外)
faqs	status='deleted', valid=0	DELETE	title / body 空字
question_logs	valid=0	DELETE	user_question 空字, ip_address / user_agent NULL, 集計値維持
inquiries	valid=0	DELETE	user_question 空字, inquiry_code 維持, closed_reason 空字
inquiry_contacts	valid=0	DELETE	email_encrypted → anon_<sha256>, display_name → 「匿名」
chat_rooms	valid=0	DELETE	(匿名化対象外、chat_messages 単位で匿名化)

対象テーブル	論理削除 (v1.10 新設)	物理削除 (90 日後バッチ or 即時、 data_deletion_mode='physical')	匿名化 (data_deletion_mode='anonymize')
chat_messages	(valid なし、 chat_rooms.valid=0 で 十分)	DELETE	content 空字, actor_type 維持
billing_subscriptions	valid なし	(物理削除バッチ対象外、永久保持)	(匿名化対象外、電子帳簿保存法)
notification_logs	(valid なし、ログ系で論理 削除概念外)	DELETE	recipient_email_hmac → SHA-256, vars → 空 JSON
access_tokens	(valid なし、used_at で代 用)	DELETE	(常に物理削除、匿名化対象外)
sessions	(valid なし、revoked_at で 代用)	DELETE	(匿名化対象外)
audit_logs	(valid なし、tombstone 方 式)	削除しない	(tombstone 方式、匿名化対象外)

1.9.2 9.2 論理削除の物理削除バッチ (v1.10、DD14 正本)

- ・ 日次バッチで `valid=0 AND updated_at < now() - 90d` の行を物理削除モード (`accounts.data_deletion_mode` 準拠) に従って処理
- ・ `accounts` 行が物理 DELETE された時点で `ON DELETE CASCADE` で関連 `account_project_grants / sessions` 等が連鎖物理削除
- ・ `projects` 行が物理 DELETE された時点で `ON DELETE CASCADE` で関連 `faqs / inquiries` 等が連鎖物理削除
- ・ `billing_subscriptions` は物理削除バッチの対象外 (永久保持)

1.10 10. データ保持の調整制約 (NFR-702)

- ・ データ保持期間はシステム固定値とし、契約・管理者ユーザーによる UI 上の変更は不可 (短縮・長期化いずれも不可)
- ・ 固定値は本書 §2 のテーブル一覧「保持期間」列および要件定義 NFR-701 / NFR-705 を正本とする
- ・ 短縮が必要な場合 (法令・契約義務範囲内) はサポート窓口経由の運営者対応とし、長期化が必要な場合は将来要件 (FUT01_認証・セキュリティ強化 / FUT 配下) で再検討する
- ・ SCR-028 アカウント設定にデータ保持期間設定 UI を持たない (基本設計 01_画面設計.md §5.20b 参照。SCR-016 は v1.5 で SCR-028 に統合・廃止)
- ・ `usage_metering / billing_invoices / audit_logs(retention_class='billing')` は電子帳簿保存法・国税法の要請により最低 7 年保持 (NFR-602b)、これを下回る短縮は不可

1.11 11. PII 取扱 (§7.10 から移管)

観点	仕様
暗号化方式	AES-256-GCM 列単位、オーナー派生 (NFR-319)
対象テーブル	<code>inquiry_contacts.email_encrypted</code> 、 <code>accounts.email_encrypted</code>
検索方式	暗号化 + *_hmac カラム (HMAC-SHA256) で復号不要
ローテーション	年次 Master Key ローターション時に KDF 再実行。60 日 dual-decrypt 期間 (NFR-321)
カード情報	Stripe に委譲 (PCI-DSS 範囲外)

詳細は [10_セキュリティ設計書.md §3 / §8](#) を正本とする。

1.12 12. インデックス方針 (§7.6 から移管)

観点	方針
FAQ 検索 (FTS5)	<code>faq_search_fts</code> 仮想テーブル、(<code>project_id</code> , <code>status='published'</code>) 前段フィルタ
未解決質問一覧	(<code>owner_account_id</code> , <code>project_id</code> , <code>case_status</code> , <code>created_at</code> DESC)
お知らせ一覧 (NFR-106)	(<code>recipient_account_id</code> , <code>read_at</code>) / (<code>recipient_account_id</code> , <code>category</code> , <code>created_at</code> DESC)
未読件数 (NFR-106)	KV キャッシュ <code>inbox:unread:<account_id></code> (TTL 60 秒) + DB フォールバック

観点	方針
ユニーク	inquiry_code, widget_public_key, access_tokens.token_hash, (owner_account_id, billing_year_month)
監査ログ	(owner_account_id, action, created_at) (WHERE tombstone=0), (retention_class, created_at)(保持期間バッチ用)

1.13 13. 未決事項

No	内容	確認先	期限	ステータス
1	詳細設計書の inquiries.case_status に CHECK 制約欠落(本書では補正済)	詳細設計書 行 6060	-	補正待ち
2	rate_limit_overrides / budget_limit_overrides を別テーブルに分離するか(現状 owner_quota_overrides.resource_kind で統一)	-	-	既決: 統一実装で継続
3	data_deletion_mode を accounts の独立カラムにするか(現状 notification_preferences JSON 内)	詳細設計書 § 7.8	—	完了 (v1.6 で独立カラム化、notification_preferences 列は DROP)

1.14 14. 変更履歴

日付	版数	変更内容	変更者
2026-05-17	1.0	初版作成	claude
2026-05-19	1.6	accounts.notification_preferences(JSON) 列を DROP。data_deletion_mode を独立カラム(TEXT, CHECK physical/anonymize) として昇格。プロジェクト通知設定をメンバー選択型に再正規化するため project_notification_settings(§ 3.8b、1対1)と project_notification_recipients(§ 3.8c、project_id × severity × account_id の3軸正規化)を新設(normal/lowのみ)。critical は共有概念に基づき強制送信のためテーブルに保持しない。テーブル一覧に 8b/8c 行を追加。メンバー離脱・完全削除時の自動除外は API 層 + ON DELETE CASCADE で担保	claude
2026-05-19	1.7	FR-015d の「メンバーは常に最低1件の割当を保持する不変条件」を撤廃。v1.7 でメンバー削除を アカウント全体の物理削除 (DELETE /projects/{id}/members/{accountId}) に統一し、account_project_grants 行のみの単独削除(離脱)操作を廃止したため、割当0件のメンバーアカウントは構造的に発生しなくなった。最後の admin が削除された場合はオーナーが暗黙の管理者として継承 (FR-019a) するため、account_project_grants の admin 件数を1以上に保つ DB スキーマ制約は不要。accounts 行・全 account_project_grants 行・全 sessions 行・全 project_notification_recipients 行を ON DELETE CASCADE で連鎖物理削除(全プロジェクトの宛先選択も自動除外)	claude
2026-05-19	1.8	SCR-030 簡素化に伴うスキーマ変更。project_notification_settings(§ 3.8b)を再設計——delivery_schedule / daily_digest_at 列を削除(配信は常に即時)、notification_enabled INTEGER(0/1)列を新設(プロジェクト単位のオン/オフトグル)。project_notification_recipients(§ 3.8c)を廃止——配信先はトグル ON 時に当該プロジェクトの account_project_grants を動的スキャンしてオーナー + admin + member 全員に解決する方式に変更。メンバーアカウント物理削除時の連鎖削除リストから project_notification_recipients を除外(全 account_project_grants 行・全 sessions 行のみで足りる)	claude
2026-05-19	1.9	SCR-030 再設計に伴うスキーマ変更。project_notification_settings(§ 3.8b)の notification_enabled / faq_candidate_threshold 列を削除、chat_response_notification_enabled INTEGER(0/1)列を新設(チャット応対通知グループのトグル)。「同一質問繰り返し N 回で自動 FAQ 候補化」機能本体を廃止したため faq_candidate_threshold 列が不要となった (inquiries.faq_candidate_state 列は手動操作経路として維持)。将来通知グループ追加時は同テーブルに *_notification_enabled 列を追加する命名規約	claude
2026-05-19	2.0	SCR-030 プロジェクト設定画面を完全廃止したことに伴い、project_notification_settings(§ 3.8b)テーブルごと廃止。プロジェクト関連通知は常時 ON 固定となり、画面設定値を保持する必要がなくなった。配信先解決は配信時点で account_project_grants を動的スキャンする方式を継続	claude

日付	版数	変更内容	変更者
2026-05-19	1.10	オーナー固定化+論理削除統一(validカラム導入)+プロジェクト削除動線 SCR-026集約に伴うスキーマ変更。① § 2.3/ § 3.1/ § 3.2で「オーナーはaccount_project_grants行を持たない」設計を撤回し、プロジェクト作成時にaccount_project_grants(role='admin', valid=1)行を自動INSERTする方式に統一。② 11 テーブル(accounts / account_project_grants / projects / allowed_domains / project_ip_allowlist / faqs / question_logs / inquiries / inquiry_contacts / chat_rooms / billing_subscriptions)に valid INTEGER NOT NULL DEFAULT 1 CHECK (valid IN (0,1)) カラムを追加。deleted_at カラムは追加せず、論理削除日時は既存 updated_at で代用。③ uq_accounts_owner_email を WHERE valid=1 部分 UNIQUE に変更(論理削除済みアカウントとの同メール再招待を許容)。④ § 9 削除モード表に「論理削除」モードを第3のモードとして新設、90 日後物理削除バッチ (DD14) を導入 (billing_subscriptions は除外、永久保持)。⑤ § 3.2 「プロジェクト削除時の孤立メンバー処理」を物理 → 論理削除化 (UPDATE … SET valid=0)	claude